Guia de Integração de Recrutas

Git & GitHub numa Organization + Exercício com Raspberry Pi Pico 2 W (TSAL e Brake Light)

NOME DA EQUIPA – Formula Student

26 de outubro de 2026

Conteúdo

1 Introdução **2**

2 Parte 1 – Git e GitHub Desktop **2**
2.1 Conceitos essenciais . . . 2
2.2 Preparação (uma única vez) . . . 2
2.3 Fluxo de trabalho no GitHub Desktop (passo a passo) . . . 3
2.4 Equivalente em linha de comandos . . . 3
2.5 Boas regras numa Organization . . . 4
2.6 Proteções recomendadas para os administradores . . . 4
2.7 Estrutura de repositório sugerida . . . 5

3 Parte 2 – Exercício: TSAL ou Brake Light com o Pico 2 W **5**
3.1 Contexto: o que são o TSAL e a Brake Light? . . . 5
3.2 Divisão dos recrutas . . . 5
3.3 Material necessário . . . 5
3.4 Pinout e ligações . . . 6
3.5 Preparar o Arduino IDE para o Pico 2 W . . . 6
3.6 Passo 0 – Teste "Hello World" (Blink) . . . 6
3.7 Ficheiro comum: `pins.h` . . . 7
3.8 Grupo A – Exercício TSAL . . . 7
3.9 Grupo B – Exercício Brake Light . . . 8
3.10 Critérios de aceitação (ambos os grupos) . . . 10

4 Alternativa: VS Code + PlatformIO **10**
4.1 Instalação . . . 10
4.2 Criar o projeto . . . 10
4.3 Ficheiro `platformio.ini` . . . 11
4.4 Código em `src/main.cpp` . . . 11
4.5 Compilar, carregar e monitorizar . . . 11
4.6 Integração com Git no VS Code . . . 11
4.7 Arduino IDE vs. PlatformIO . . . 12

5 Extras importantes **12**
5.1 Segurança em eletrónica e no workshop . . . 12
5.2 Boas práticas de código para sistemas de segurança . . . 12
5.3 Debug: problemas comuns . . . 12
5.4 Checklist final do recruta . . . 13
5.5 Para saber mais . . . 13

1 Introdução

Este documento tem duas partes:

Parte 1. Como trabalhar em equipa com **Git**, **GitHub Desktop** e uma **Organization** do GitHub, seguindo boas regras.

Parte 2. Um exercício prático com o **Raspberry Pi Pico 2 W**, onde cada recruta irá simular o **TSAL** (*Tractive System Active Light*) ou a **Brake Light** do nosso monoposto, usando **Arduino IDE** (e, em alternativa, **VS Code + PlatformIO**).

No fim do exercício, cada recruta deve ter entregue o seu código através de um *Pull Request* no repositório da equipa — é a forma de juntar as duas partes.

Atenção

Este exercício é **didático**. Na competição, as regras (Formula Student Germany / Portugal, conforme o evento) impõem requisitos específicos ao TSAL e à brake light, incluindo que o TSAL **não pode depender de software** (implementação em *hardware* dedicado). Consultem sempre o regulamento em vigor. O objetivo aqui é compreender a *lógica* e treinar boas práticas de código.

2 Parte 1 – Git e GitHub Desktop

2.1 Conceitos essenciais

Repositório (*repo*)
Pasta de projeto com todo o histórico de alterações.

Organization Conta partilhada no GitHub onde vivem os repositórios da equipa, com membros, equipas (*teams*) e permissões.

Commit "Fotografia" do projeto num momento, com uma mensagem a explicar o que mudou.

Branch Linha de desenvolvimento paralela. Permite trabalhar sem estragar o código dos outros.

Pull Request (PR)
Pedido para juntar o teu *branch* ao `main`, sujeito a revisão.

Clone / Fetch / Pull / Push
Copiar o repo / ver o que mudou online / trazer alterações / enviar as tuas.

2.2 Preparação (uma única vez)

1. Criar conta em `https://github.com` (usar um nome reconhecível).
2. Ativar a **autenticação de dois fatores (2FA)** — é normalmente obrigatória em Organizations.
3. Enviar o *username* ao responsável da equipa para ser convidado para a Organization `nome-da-organizatio`
   Aceitar o convite recebido por email.
4. Instalar o **GitHub Desktop** (`https://desktop.github.com`) e fazer *Sign in* com a conta GitHub.

5. Instalar o **Git** (o GitHub Desktop já o inclui; para o terminal/VS Code instalar também de https://git-scm.com).

Dica

No GitHub Desktop: *File* → *Options* → *Git* e confirmar que o **nome** e o **email** estão corretos. Aparecem em todos os teus commits.

2.3 Fluxo de trabalho no GitHub Desktop (passo a passo)

1. Clonar o repositório. *File* → *Clone repository* → *separador GitHub.com* e escolher `nome-da-organizatio` Escolher uma pasta local **sem espaços nem acentos** no caminho, por exemplo `C:\dev\`.

2. Atualizar antes de começar. Todos os dias, antes de mexer em código: *Fetch origin* e, se aparecer, *Pull origin*. Assim começas sempre com a versão mais recente do `main`.

3. Criar um branch. *Current Branch* → *New Branch*. Nome segundo a convenção da secção 2.5, por exemplo `feature/joao-tsal-blink`. Garantir que o branch é criado a partir do `main`.

4. Trabalhar e fazer commits pequenos. Editar os ficheiros no Arduino IDE / VS Code. No GitHub Desktop, o separador *Changes* mostra o que mudou. Seleciona os ficheiros, escreve o **Summary** (mensagem curta) e clica *Commit to* `<branch>`.

5. Enviar o branch. *Publish branch* (primeira vez) ou *Push origin* (seguintes).

6. Abrir um Pull Request. *Branch* → *Create Pull Request* abre o GitHub no browser. Preencher título e descrição (o que fizeste, como testaste) e pedir revisão a um colega ou ao responsável de subsistema.

7. Rever, corrigir e *merge*. Se pedirem alterações, fazer novos commits no **mesmo branch** — o PR atualiza-se sozinho. Quando aprovado, o *merge* é feito no GitHub (normalmente *Squash and merge*). Depois: voltar ao `main` no Desktop, *Pull origin* e apagar o branch antigo.

Resolver conflitos. Se o Desktop avisar de conflitos, abrir os ficheiros no VS Code (o Desktop oferece *Open in Visual Studio Code*), escolher o código correto entre as marcas `<<<<<<<` / `=======` / `>>>>>>>`, remover as marcas, guardar e fazer commit. Em caso de dúvida, **perguntar** antes de apagar código de outra pessoa.

2.4 Equivalente em linha de comandos

Útil para quem prefere o terminal (ou para o terminal integrado do VS Code):

```
git clone https://github.com/ORGNAME/pico-tsal-brakelight.git
cd pico-tsal-brakelight
git switch main && git pull                # atualizar
git switch -c feature/joao-tsal-blink      # novo branch
git status                                 # ver alteracoes
git add src/tsal.ino                       # preparar ficheiros
git commit -m "feat(tsal): pisca vermelho a 4 Hz sem delay()"
git push -u origin feature/joao-tsal-blink
```

2.5 Boas regras numa Organization

Regra de ouro

Nunca fazer commit diretamente no `main`. Todo o código entra por *Pull Request*, revisto por pelo menos uma outra pessoa.

1. **Um branch por tarefa.** Nomes: `feature/`, `fix/`, `docs/` seguido de `nome-descricao-curta` em minúsculas e com hífenes.
2. **Commits pequenos e frequentes**, cada um com uma ideia só.
3. **Mensagens de commit claras**, no formato `tipo(âmbito): descrição no imperativo`. Tipos comuns: `feat`, `fix`, `docs`, `refactor`, `test`, `chore`. Exemplo: `fix(brake): adicionar histerese ao limiar de pressão`. Evitar "alterações", "coisas", "final2".
4. **Pull antes de começar** e **push no fim do dia**: trabalho só no teu computador não existe para a equipa (nem sobrevive a um disco avariado).
5. **PRs pequenos** (idealmente < 300 linhas) e com descrição: o quê, porquê, como testar.
6. **Revisão construtiva.** Rever é aprender. Comentar o código, não a pessoa. Aprovar só o que percebeste.
7. **Não versionar lixo nem segredos.** Usar um `.gitignore` (pastas de *build*, `.pio/`, `.vscode/` pessoal, ficheiros `.uf2` gerados). **Nunca** fazer commit de palavras-passe, *tokens* ou credenciais Wi-Fi.
8. **Ficheiros binários grandes** (CAD, PDFs pesados, datalogs) não pertencem ao Git: usar a drive da equipa ou Git LFS, conforme indicado pelo responsável.
9. **Issues e Projects.** Cada tarefa tem uma *Issue*; o PR referencia-a (`Closes #12`). Assim o histórico fica rastreável.
10. **Documentar.** Um `README.md` em cada subsistema: o que faz, ligações de hardware, como compilar.
11. **Nunca reescrever histórico partilhado** (`git push -force` no `main` ou em branches de outros). Em caso de erro, pedir ajuda.
12. **Licenças e dados.** Não colocar código copiado sem referência; indicar a origem.

2.6 Proteções recomendadas para os administradores

Em *Settings* → *Branches* → *Branch protection rules* para o `main`:

- Exigir *Pull Request* antes do merge e pelo menos **1 aprovação**;
- Exigir que a discussão esteja resolvida;
- Bloquear *force push* e a eliminação do branch;
- Opcional: ficheiro `CODEOWNERS` para atribuir revisores automáticos por subsistema;
- Opcional: *GitHub Actions* a compilar o código em cada PR (por exemplo com PlatformIO).

2.7 Estrutura de repositório sugerida

```
pico-tsal-brakelight/
|-- README.md
|-- .gitignore
|-- docs/                 # esquemas, fotos de montagem, pinout
|-- arduino/
|   |-- tsal/tsal.ino
|   `-- brakelight/brakelight.ino
`-- platformio/
    |-- platformio.ini
    `-- src/main.cpp
```

3 Parte 2 – Exercício: TSAL ou Brake Light com o Pico 2 W

3.1 Contexto: o que são o TSAL e a Brake Light?

TSAL (*Tractive System Active Light*). Luz visível à volta do carro que indica o estado do sistema de tração (alta-tensão). Numa abordagem típica do regulamento:

- **Vermelho intermitente** (cerca de 2–5 Hz) quando o sistema de tração está **ativo** (tensão do acumulador acima de 60 V);
- **Verde fixo** quando o sistema está desligado e é seguro tocar no carro.

É uma questão de **segurança humana**: quem vê vermelho a piscar não toca no carro.

Brake Light. Luz vermelha traseira que **acende quando o travão é acionado**, informando quem vem atrás. A deteção é feita normalmente por um sensor de pressão do circuito de travagem (ou um interruptor mecânico no pedal).

Atenção

Os valores acima são indicativos. Confirmar frequências, cores e requisitos no regulamento da edição em causa. No carro real, a **lógica de segurança** do TSAL é feita por circuito dedicado; o Pico serve aqui para aprender e prototipar.

3.2 Divisão dos recrutas

	Grupo A – TSAL	**Grupo B – Brake Light**
Entrada	Botão que simula "TS ativo" (>60 V)	Botão (pedal) *ou* potenciómetro (sensor de pressão)
Saída	LED vermelho (pisca) e LED verde (fixo)	LED vermelho (acende ao travar)
Conceitos	Estados, temporização sem `delay()`, *debounce*	ADC, limiar com histerese, falha segura

3.3 Material necessário

- 1× Raspberry Pi Pico 2 W (com pinos soldados) + cabo micro-USB;
- **Grupo A:** 1 LED vermelho, 1 LED verde, 2 resistências de 330 Ω, 1 botão de pressão;
- **Grupo B:** 1 LED vermelho, 1 resistência de 330 Ω, 1 botão *ou* 1 potenciómetro de 10 kΩ;
- Breadboard e jumpers.

3.4 Pinout e ligações

O pinout será fornecido pelo responsável da equipa. Preencher a tabela abaixo (e as constantes no código, ficheiro `pins.h`).

Sinal	Pino (GPIO)	Ligação	Grupo
`PIN_TS_BUTTON`	GP___ (a preencher)	Botão → GND (usa `INPUT_PULLUP`)	A
`PIN_LED_RED`	GP___ (a preencher)	GPIO → 330 Ω → LED → GND	A, B
`PIN_LED_GREEN`	GP___ (a preencher)	GPIO → 330 Ω → LED → GND	A
`PIN_BRAKE_BTN`	GP___ (a preencher)	Botão → GND (`INPUT_PULLUP`)	B
`PIN_BRAKE_ADC`	GP26/27/28 (ADC)	Cursor do potenciómetro; extremos em 3V3 e GND	B (opcional)

Atenção

- Os GPIO do Pico trabalham a **3,3 V** e **não toleram 5 V**. Um sensor com saída até 5 V (típico em sensores de pressão automóvel) exige **divisor de tensão**.
- Todos os LEDs levam **resistência em série**. Sem ela, queimam o LED e/ou o pino.
- Montar sempre com o USB **desligado**. Só ligar depois de o professor/colega verificar.
- Apenas os pinos GP26, GP27 e GP28 têm ADC. Confirmar no *pinout* oficial.

3.5 Preparar o Arduino IDE para o Pico 2 W

1. Instalar o **Arduino IDE 2.x** (https://www.arduino.cc/en/software).
2. *File* → *Preferences* → *Additional boards manager URLs* e adicionar:

```
https://github.com/earlephilhower/arduino-pico/releases/download/global/
    package_rp2040_index.json
```

3. *Tools* → *Board* → *Boards Manager*, pesquisar **"Raspberry Pi Pico/RP2040/RP2350"** (autor: Earle F. Philhower, III) e instalar.
4. Selecionar *Tools* → *Board* → *Raspberry Pi Pico/RP2040/RP2350* → *Raspberry Pi Pico 2 W*.
5. **Primeiro upload:** desligar o USB, **manter premido o botão BOOTSEL**, ligar o USB e largar o botão. O Pico aparece como unidade de disco (`RP2350`). Selecionar a porta e carregar em *Upload*. Nos uploads seguintes, a porta COM aparece automaticamente.
6. Abrir o *Serial Monitor* (115200 baud) para ver mensagens de *debug*.

Dica

Se o Pico não aparece como porta COM, usar outro cabo (muitos cabos são só de carga) e repetir o procedimento com BOOTSEL.

3.6 Passo 0 – Teste "Hello World" (Blink)

Antes de qualquer lógica, confirmar que a cadeia (IDE → placa) funciona. No Pico 2 W o LED integrado está ligado ao chip Wi-Fi, mas o núcleo Arduino mapeia-o em `LED_BUILTIN`:

```ino
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(500);
  digitalWrite(LED_BUILTIN, LOW);
  delay(500);
}
```

3.7 Ficheiro comum: `pins.h`

Todas as ligações num só sítio, para alterar facilmente (**substituir os números pelo pinout fornecido**):

```ino
// pins.h -- ligacoes do Pico 2 W
#pragma once

constexpr uint8_t PIN_TS_BUTTON = 0;   // TODO: preencher (Grupo A)
constexpr uint8_t PIN_LED_RED   = 1;   // TODO: preencher
constexpr uint8_t PIN_LED_GREEN = 2;   // TODO: preencher (Grupo A)
constexpr uint8_t PIN_BRAKE_BTN = 3;   // TODO: preencher (Grupo B)
constexpr uint8_t PIN_BRAKE_ADC = 26;  // GP26 = ADC0 (Grupo B, opcional)
```

3.8 Grupo A – Exercício TSAL

Objetivo. Simular o TSAL. Quando o botão "TS ativo" está premido, o LED vermelho pisca a 4 Hz e o verde apaga. Quando não está, o verde fica fixo e o vermelho apagado.

Lógica (máquina de estados).

Estado	Condição	Saídas
`TS_OFF`	botão solto	Verde ON, Vermelho OFF
`TS_ON`	botão premido	Verde OFF, Vermelho a piscar (4 Hz ⇒ 125 ms ON / 125 ms OFF)

Ideias a ensinar.

- **Nunca usar `delay()`** para o piscar: bloqueia o processador e, num sistema real, impediria a leitura de sensores. Usar `millis()`.
- **Debounce:** o botão "salta" alguns milissegundos ao ser premido. Filtrar com um tempo mínimo estável.
- `INPUT_PULLUP`: o botão liga a GND, logo **premido = LOW**.

Código de referência.

```ino
#include "pins.h"

constexpr uint32_t BLINK_HALF_PERIOD_MS = 125;  // 4 Hz
constexpr uint32_t DEBOUNCE_MS          = 30;

enum class TsState { OFF, ON };
TsState state = TsState::OFF;

// --- debounce ---
bool stableLevel = HIGH;      // nivel estavel (pullup: solto = HIGH)
bool lastReading = HIGH;
```

```cpp
uint32_t lastChangeMs = 0;

// --- pisca ---
uint32_t lastToggleMs = 0;
bool redLevel = false;

bool tsButtonPressed() {
  bool reading = digitalRead(PIN_TS_BUTTON);
  if (reading != lastReading) {
    lastChangeMs = millis();
    lastReading = reading;
  }
  if (millis() - lastChangeMs >= DEBOUNCE_MS) {
    stableLevel = reading;
  }
  return stableLevel == LOW;   // premido = LOW
}

void setup() {
  Serial.begin(115200);
  pinMode(PIN_TS_BUTTON, INPUT_PULLUP);
  pinMode(PIN_LED_RED, OUTPUT);
  pinMode(PIN_LED_GREEN, OUTPUT);
}

void loop() {
  TsState newState = tsButtonPressed() ? TsState::ON : TsState::OFF;

  if (newState != state) {
    state = newState;
    Serial.println(state == TsState::ON ? "TS ATIVO" : "TS DESLIGADO");
  }

  if (state == TsState::OFF) {
    digitalWrite(PIN_LED_GREEN, HIGH);
    digitalWrite(PIN_LED_RED, LOW);
    redLevel = false;
  } else {
    digitalWrite(PIN_LED_GREEN, LOW);
    if (millis() - lastToggleMs >= BLINK_HALF_PERIOD_MS) {
      lastToggleMs = millis();
      redLevel = !redLevel;
      digitalWrite(PIN_LED_RED, redLevel);
    }
  }
}
```

Desafios extra (Grupo A).

1. Ler com um **potenciómetro** uma "tensão do acumulador" simulada (0–600 V mapeados no ADC) e ativar o TSAL vermelho quando for > 60 V.
2. Acrescentar histerese (ativar acima de 60 V, desativar abaixo de 55 V).
3. Imprimir no Serial o estado e a "tensão".
4. Acender o LED do Pico (`LED_BUILTIN`) como indicador de "sistema vivo" (*heartbeat*).

3.9 Grupo B – Exercício Brake Light

Objetivo. Acender o LED vermelho quando o travão é acionado, primeiro com um botão (simples) e depois com um sensor analógico (realista).

Versão 1 – botão (digital).

```ino
#include "pins.h"

void setup() {
  pinMode(PIN_BRAKE_BTN, INPUT_PULLUP);
  pinMode(PIN_LED_RED, OUTPUT);
}

void loop() {
  bool braking = (digitalRead(PIN_BRAKE_BTN) == LOW);  // premido = LOW
  digitalWrite(PIN_LED_RED, braking ? HIGH : LOW);
}
```

Versão 2 – sensor de pressão (analógico) com histerese e falha segura. O sensor de pressão dá uma tensão proporcional à pressão. Como o sinal tem ruído, usa-se **histerese**: acende acima de um limiar alto e só apaga abaixo de um limiar mais baixo, evitando que o LED "tremule". Além disso, aplica-se **falha segura**: se a leitura estiver fora do intervalo plausível (cabo cortado, curto-circuito), a luz acende.

```ino
#include "pins.h"

// ADC de 12 bits: 0..4095 corresponde a 0..3.3 V
constexpr int ADC_BITS     = 12;
constexpr int THRESH_ON    = 900;    // acende acima deste valor
constexpr int THRESH_OFF   = 700;    // apaga abaixo deste valor
constexpr int ADC_MIN_OK   = 100;    // abaixo = sensor/cabo em falha
constexpr int ADC_MAX_OK   = 4000;   // acima  = sensor/cabo em falha

bool brakeOn = false;

void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED_RED, OUTPUT);
  analogReadResolution(ADC_BITS);
}

void loop() {
  int raw = analogRead(PIN_BRAKE_ADC);

  bool sensorFault = (raw < ADC_MIN_OK) || (raw > ADC_MAX_OK);

  if (sensorFault) {
    brakeOn = true;                          // falha segura: luz acesa
  } else if (!brakeOn && raw > THRESH_ON) {
    brakeOn = true;
  } else if (brakeOn && raw < THRESH_OFF) {
    brakeOn = false;
  }

  digitalWrite(PIN_LED_RED, brakeOn ? HIGH : LOW);

  Serial.print("raw=");   Serial.print(raw);
  Serial.print(" brake="); Serial.print(brakeOn);
  Serial.print(" fault="); Serial.println(sensorFault);
  delay(20);                                 // 50 Hz de amostragem
}
```

Dica

Com o potenciómetro, rodar o cursor até ao extremo (0 ou 3,3 V) provoca a "falha" de propósito — é uma boa forma de testar a falha segura. Num sensor real de 0,5–4,5 V (com divisor), a gama plausível é a que estiver na ficha técnica.

Desafios extra (Grupo B).

1. Calcular a média de 8 leituras do ADC para reduzir o ruído.
2. Converter o valor do ADC em **bar** com a equação do sensor e mostrá-la no Serial.
3. Fazer a luz de travão piscar 3 vezes ("travagem forte") quando a pressão passa um segundo limiar.
4. Implementar um módulo com função `bool isBraking()` separado do `loop()`.

3.10 Critérios de aceitação (ambos os grupos)

- O código compila sem avisos relevantes e funciona na placa.
- Não há `delay()` que bloqueie a lógica principal (exceto onde indicado).
- Pinos definidos em constantes (`pins.h`), sem "números mágicos".
- Comentários úteis e nomes de variáveis claros.
- O trabalho está num **branch**, com commits claros, e entregue por **Pull Request**.
- O PR inclui uma foto da montagem e uma breve descrição de como testar.

4 Alternativa: VS Code + PlatformIO

O Arduino IDE é ótimo para começar. Para projetos maiores (vários ficheiros, bibliotecas, integração com Git, *autocomplete*), o **PlatformIO** dentro do **VS Code** é mais adequado — e é o que recomendamos para código partilhado pela equipa.

4.1 Instalação

1. Instalar o **Visual Studio Code** (`https://code.visualstudio.com`).
2. Separador *Extensions* (`Ctrl+Shift+X`), pesquisar **PlatformIO IDE** e instalar. Aguardar que termine a instalação e reiniciar o VS Code.
3. Aparece o ícone da formiga (PlatformIO) na barra lateral e a barra azul no fundo.

4.2 Criar o projeto

1. *PIO Home* → *New Project*.
2. Nome: `tsal-pico2w`. Board: procurar **"Raspberry Pi Pico 2 W"** (`rpipico2w`). Framework: **Arduino**.
3. O PlatformIO cria `platformio.ini`, a pasta `src/` e `lib/`.

4.3 Ficheiro `platformio.ini`

Para o Pico 2 W com o núcleo Arduino-Pico (o mesmo do Arduino IDE), usar uma configuração semelhante a esta:

```
[env:pico2w]
platform  = https://github.com/maxgerhardt/platform-raspberrypi.git
board     = rpipico2w
framework = arduino
board_build.core = earlephilhower
monitor_speed    = 115200
upload_protocol  = picotool
; alternativa sem picotool: upload manual por BOOTSEL (copiar o .uf2)
```

Dica

O suporte a RP2350/Pico 2 no PlatformIO depende da *platform* usada e evolui depressa. Se a compilação falhar, confirmar a configuração na documentação atual do núcleo Arduino-Pico e do PlatformIO. Se o upload automático não funcionar, basta pôr o Pico em modo BOOTSEL e copiar o ficheiro `.uf2` gerado em `.pio/build/pico2w/`.

4.4 Código em `src/main.cpp`

No PlatformIO o ficheiro é C++ normal, por isso é preciso incluir o Arduino:

```
#include <Arduino.h>
#include "pins.h"        // colocar pins.h dentro de include/

void setup() {
  // ... igual ao exemplo do Arduino IDE ...
}

void loop() {
  // ... igual ao exemplo do Arduino IDE ...
}
```

4.5 Compilar, carregar e monitorizar

Na barra azul inferior (ou na formiga → *Project Tasks*):

Ação	Atalho / ícone
Build (compilar)	ícone (`Ctrl+Alt+B`)
Upload (carregar)	ícone → (`Ctrl+Alt+U`)
Serial Monitor	ícone ficha (`Ctrl+Alt+S`)
Clean	ícone caixote

4.6 Integração com Git no VS Code

O VS Code tem Git integrado (separador *Source Control*, `Ctrl+Shift+G`): mostra alterações, permite *stage*, *commit*, *push* e criar branches. Pode ser usado em conjunto com o GitHub Desktop — ambos trabalham sobre o mesmo repositório local.

Ficheiro `.gitignore` para PlatformIO:

```
.pio/
.vscode/.browse.c_cpp.db*
.vscode/c_cpp_properties.json
.vscode/launch.json
.vscode/ipch
*.uf2
```

4.7 Arduino IDE vs. PlatformIO

	Arduino IDE	**VS Code + PlatformIO**
Curva de aprendizagem	Muito baixa	Média
Autocomplete / navegação	Básico	Muito bom
Gestão de bibliotecas	Manual / Library Manager	Declarada no `platformio.ini` (reprodutível)
Git integrado	Não	Sim
Vários ficheiros / projetos grandes	Limitado	Excelente
Ideal para	Primeiros passos, testes rápidos	Código da equipa, CI, projetos maiores

5 Extras importantes

5.1 Segurança em eletrónica e no workshop

- Verificar a montagem antes de ligar a alimentação.
- Nunca ligar saídas de GPIO diretamente a cargas de potência: usar MOSFET/transístor e díodo de *flyback* se a carga for indutiva. Uma lâmpada de brake light real consome muito mais do que os pinos aguentam (cerca de 12 mA por pino).
- Em qualquer trabalho com o acumulador/alta-tensão: **só** com formação e supervisão do responsável de segurança. Este exercício é feito a 3,3 V/USB.

5.2 Boas práticas de código para sistemas de segurança

1. **Falha segura (*fail-safe*)**: perante dúvida ou erro, o estado seguro (brake light acesa, TSAL a indicar perigo).
2. **Sem bloqueios**: evitar `delay()` e ciclos infinitos na lógica de segurança.
3. **Constantes nomeadas** para limiares, tempos e pinos.
4. **Funções pequenas** e com uma única responsabilidade.
5. **Testar os casos limite**: sensor desligado, valores extremos, botão preso.
6. **Watchdog**: reinicia o microcontrolador se o programa encravar (no Arduino-Pico: `rp2040.wdt_begin(ms)` e `rp2040.wdt_reset()` no `loop()`).

5.3 Debug: problemas comuns

Sintoma	**Causa provável**
Porta COM não aparece	Cabo só de carga; falta BOOTSEL no 1.º upload
LED não acende	LED ao contrário (polaridade); pino errado; sem resistência/GND
Botão "lê" ao contrário	`INPUT_PULLUP`: premido = `LOW`
Botão dispara várias vezes	Falta *debounce*
Leituras ADC instáveis	Ruído; fazer média; verificar GND comum; pino que não é ADC
Erro ao compilar	Placa errada selecionada; núcleo desatualizado; falta `#include`

5.4 Checklist final do recruta

Conta GitHub com 2FA e convite da Organization aceite.

GitHub Desktop instalado e repositório clonado.

Arduino IDE configurado e Blink a funcionar no Pico 2 W.

Montagem verificada com a tabela de pinout.

Código do TSAL / Brake Light a funcionar.

Branch criado, commits claros e Pull Request aberto.

PR com descrição, foto da montagem e instruções de teste.

5.5 Para saber mais

- Documentação Git: https://git-scm.com/doc
- GitHub Desktop: https://docs.github.com/desktop
- Núcleo Arduino-Pico: https://arduino-pico.readthedocs.io
- Raspberry Pi Pico 2 W (datasheet e pinout): https://www.raspberrypi.com/documentation/microcontrollers/
- PlatformIO: https://docs.platformio.org
- Regulamento Formula Student da edição em causa (TSAL, brake light e requisitos de segurança)